TECHNICAL ADVISORY & PROPOSAL

Solving Production Real-Time Messaging Issue (Laravel Reverb)

Project:	OnePAD Team Recognition Platform
Target Audience:	Project Manager & DevOps Team
Advisory Type:	Production Infrastructure Adjustment
Prepared by:	Technical Engineering Team

---

1. Executive Summary

During local development, our real-time messaging system (including instant typing indicators, seen status updates, real-time message edits/deletions, and instant calls) works flawlessly. However, in the hosted production environment, messages only appear after a manual browser page reload.

This diagnostic behavior confirms that the **database persistence, Laravel controllers, and standard HTTP API request flows are 100% operational**. The root cause lies entirely in the real-time broadcast and WebSocket infrastructure (Laravel Reverb) which is failing to establish or receive WebSocket connections in production.

This document details the exact reasons for this discrepancy and provides a safe, standard production configuration to resolve the issue completely without making any changes to our React or PHP codebase.

2. Technical Root Causes

There are three primary operational differences between local development and production environments that prevent real-time communication from functioning:

- **A. The Reverb WebSocket Server Process is Not Daemonized:** Locally, running `composer run dev` automatically spins up the WebSocket server process (`php artisan reverb:start`). In production, this command does not run in the background by default. If the server process is not active on the server, the server will not listen on port 8080.
- **B. Browser Security & Secure WebSockets (WSS):** Because the hosted production website is served over HTTPS, modern browsers strictly block any insecure connections, including insecure WebSockets (`ws://`). The production connection MUST be established over secure WebSockets (`wss://`).
- **C. Corporate/Hosting Firewall & Port Restrictions:** Reverb listens on port 8080. Directly exposing port 8080 to the public internet is typically blocked by web hosting providers for security reasons. In production, we must configure our Nginx web server as a **Reverse Proxy**. This routes secure WebSocket traffic (`wss://`) on standard HTTPS port 443 internally to port 8080, securing the connection behind Nginx.

3. Recommended Implementation Steps

Step 1: Background Daemon Setup (Supervisor)

To ensure the Reverb WebSocket server runs continuously and recovers automatically from crashes or reboots, we recommend setting up a Supervisor config file at `/etc/supervisor/conf.d/reverb.conf` on the production server:

```
[program:reverb]
process_name=%(program_name)s_%(process_num)02d
command=php /var/www/bravo/artisan reverb:start
autostart=true
autorestart=true
user=www-data
redirect_stderr=true
stdout_logfile=/var/www/bravo/storage/logs/reverb.log
stopasgroup=true
killasgroup=true
```

Step 2: Reverse Proxy Configuration (Nginx)

Add this location block inside the primary secure `server { ... }` block (port 443) of your production Nginx config to forward public WebSocket requests internally to Reverb:

```
location /app {
    proxy_http_version 1.1;
    proxy_set_header Host $http_host;
    proxy_set_header Scheme $scheme;
    proxy_set_header SERVER_PORT $server_port;
    proxy_set_header REMOTE_ADDR $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "Upgrade";

    proxy_pass http://127.0.0.1:8080;
}
```

Step 3: Server Environment Configuration (.env)

Configure your production backend and frontend environment files as follows, mapping to our secure Nginx reverse proxy endpoint:

```
# Production Laravel Backend (.env)
BROADCAST_CONNECTION=reverb

# Internal Coordinates
REVERB_SERVER_HOST=127.0.0.1
REVERB_SERVER_PORT=8080

# Public-Facing Secure Endpoint (Nginx Proxy)
REVERB_HOST=yourdomain.com
REVERB_PORT=443
REVERB_SCHEME=https

# Production React Frontend (.env) - Rebuild assets after saving!
VITE_REVERB_APP_KEY="bravo-production-key"
VITE_REVERB_HOST="yourdomain.com"
VITE_REVERB_PORT="443"
VITE_REVERB_SCHEME="https"
```

4. Solution Benefits & Next Steps

- **Zero Code Modification:** The current React and Laravel codebase requires zero changes.
- **Robust Security:** We avoid opening external ports (like 8080), routing all traffic via HTTPS/443.
- **Process Resilience:** Supervisor ensures Reverb stays running continuously without manual monitoring.

> **REQUIRED DEPLOYMENT ACTION**
>
> Once these server configurations are applied, remember to run `php artisan config:clear` on the production server, build your production frontend assets, and run `sudo supervisorctl update` to apply and initialize the daemon process. This will immediately restore full real-time messaging capabilities.